ADAMBOAS.COM • PAPER • January 25, 2025

From PDFs to Pull Requests

Code-as-Policy: Transforming Department of Defense Policy with DevSecOps

---

Adam Boas
Solutions Architect

Published January 25, 2025

Distribution: Public. **Disclaimer:** Views are the author's and do not represent official policy.

Contents

1 Introduction 3
- 1.1 Policy Management in the Digital Era . . . 3
- 1.2 Scope and Purpose of This Paper . . . 4
- 1.3 Research Objectives and Questions . . . 4

2 Theoretical and Historical Foundations 4
- 2.1 Evolution of Version Control and Collaborative Document Systems . . . 4
- 2.2 The Emergence of Docs-as-Code in Organizational Settings . . . 5
- 2.3 Code-as-Policy vs. Policy Documentation . . . 5

3 The Case for a Version-Controlled Code-as-Policy Model 5
- 3.1 Enhanced Transparency and Accountability . . . 5
- 3.2 Faster Iteration and Continuous Improvement . . . 6
- 3.3 Automation, Integration, and AI Readiness . . . 6
- 3.4 Cultural and Organizational Benefits . . . 7
- 3.5 Long-Term Strategic Alignments . . . 7

4 Potential Challenges and Mitigation Strategies 7
- 4.1 Cultural Resistance and Change Management . . . 7
- 4.2 Security and Classification . . . 8
- 4.3 Governance, Compliance, and Legal Considerations . . . 8
- 4.4 Scalability Across a Large Organization . . . 9
- 4.5 Technical Infrastructure and Interoperability . . . 10
- 4.6 User-Centric Design and Accessibility for Non-Technical Stakeholders . . . 10
- 4.7 AI Risks and Mitigations . . . 12
- 4.8 Long-Term Strategic Alignments . . . 13

5 Modernization Initiatives and the Code-as-Policy Framework 13

6 Implementation Framework 13
- 6.1 Phased Rollout . . . 13
- 6.2 Governance Model . . . 15
- 6.3 Training and Change Management . . . 16
- 6.4 Supporting Tools and Automations . . . 17

7 From Theory to Practice: Azure DevOps for Code-as-Policy 18
- 7.1 Repository Management . . . 19
- 7.2 .gitignore Configuration . . . 19
- 7.3 README.md: Onboarding and Documentation . . . 20
- 7.4 Automated Build Pipeline . . . 20
- 7.5 Deployment Pipeline . . . 23

8 Broader Impact, Real-World Examples, and Future Research 24
- 8.1 Digital Bureaucracy and Agile Governance . . . 24
- 8.2 AI-Augmented Policy and Decision Support . . . 25
- 8.3 Policy Enforcement via Machine-Readable Code . . . 26
- 8.4 Adaptive Threat Intelligence Integration . . . 26
- 8.5 Imaginative Future Directions . . . 27
- 8.6 Prioritizing Future Research . . . 28

9 On the Naming of "Code-as-Policy" 29

10 Conclusion 29

11 Glossary 30

12 Appendix: Supporting Scripts 33
- 12.1 Detailed Azure DevOps Pipeline Configurations . . . 34

Executive Summary

As the Department of Defense (DoD) intensifies its digital transformation-and increasingly acknowledges that it must operate as a software company-policy remains mired in outdated document-centric workflows. This whitepaper argues for a bold shift from static files (e.g., Word-to-PDF) to a "Code-as-Policy" (CaP) model driven by version-controlled repositories. By treating policy as dynamically managed text-much like source code-the Department of Defense can leverage the transparency, collaboration, continuous integration, and auditability that define modern software development. Drawing on interdisciplinary research in organizational theory, software engineering, digital governance, and artificial intelligence, this paper illuminates how CaP can deliver unprecedented agility, accountability, and real-time adaptability across the Department of Defense enterprise. We also examine the logistical and cultural hurdles that must be navigated-from security considerations to entrenched habits-and outline concrete strategies for scaling this practice department-wide. Ultimately, we position Code-as-Policy not merely as a technical upgrade, but as a foundational enabler of the Department of Defense's future readiness in an era defined by artificial intelligence, machine learning, cloud computing, cybersecurity, and the imperative for agility and swift adaptability. This strategic transformation not only addresses the complexities of today's technological landscape but also lays the groundwork for the DoD to seamlessly transition into the next era of autonomous systems, quantum computing, and integrated cyber-physical operations.

1 Introduction

1.1 Policy Management in the Digital Era

Policy documents in the Department of Defense are foundational instruments that inform governance, operational processes, personnel management, and strategic objectives. Over the past several decades, the Department of Defense has published instructions and directives (e.g., Department of Defense Instructions, or DoDIs) predominantly in static formats, such as Microsoft Word or PDF files, stored in centralized repositories. While such formats have been adequate for a paper-centric era, they are increasingly insufficient in an environment that demands real-time collaboration, rapid iteration, meticulous audit trails, and seamless integration with diverse systems—including emerging artificial intelligence (AI) platforms.

In parallel, commercial and public-sector organizations have adopted software-driven models for information management. Version control systems—particularly Git, but also alternatives like Perforce or Subversion—have revolutionized the way complex bodies of knowledge, including policy and documentation, are maintained and tracked. The broad concept of "Docs-as-Code" (where documentation is maintained as software in plain text and managed via version control) has demonstrated notable success in technology firms, open-source communities, and governmental agencies that require nimble processes and full traceability of changes [2]. The Department of Defense's movement toward digital modernization thus stands at a critical juncture where adopting these best practices could yield substantial benefits, while still acknowledging that Git may not be the exclusive technical solution.

1.2 Scope and Purpose of This Paper

This paper aims to provide an exhaustive, academically grounded argument for why the Department of Defense should migrate its policy management processes to a Code-as-Policy model. We will:

(i) **Review the historical and theoretical underpinnings** of Code-as-Policy models, including relevant academic and industry research.

(ii) **Explain the potential operational, strategic, and cultural advantages** that the Department of Defense stands to gain by embracing this paradigm, particularly in relation to AI and next-generation technologies.

(iii) **Address major implementation challenges**—including security, governance, and cultural shift—and propose strategies to mitigate these obstacles.

(iv) **Offer a roadmap and frameworks** for introducing and scaling a version-controlled policy management system across the Department of Defense's complex organizational structure, with an eye toward AI-readiness.

1.3 Research Objectives and Questions

This paper aims to explore the following questions:

1. How can the Code-as-Policy model enhance the agility and accountability of policy management within the Department of Defense?
2. What are the primary challenges in implementing a version-controlled policy framework, and how can they be mitigated?
3. In what ways can advanced technologies like AI and blockchain augment the effectiveness of the Code-as-Policy paradigm?

2 Theoretical and Historical Foundations

2.1 Evolution of Version Control and Collaborative Document Systems

Version control systems have been integral to software engineering since the emergence of Source Code Control System (SCCS) in the 1970s. Over time, systems like CVS, Subversion (SVN), and Git have refined the concept of distributed collaboration. Git's distributed nature and lightweight branching model have made it the *de facto* standard in modern software development [3]. However, other tools like Perforce are used for certain high-scale or specialized environments, demonstrating that version control approaches can be adapted to different organizational needs.

Key Attributes of Version Control Relevant to Policy Management:

1. *Granular Change History*: Every edit is captured with an author signature and timestamp, enabling detailed blame and audit capabilities.
2. *Branching and Merging*: Proposed changes can be isolated, reviewed, and tested in separate branches, then merged once approved.
3. *Tagging/Releases*: Official "releases" of policy can be tagged, providing clarity on recognized versions.
4. *Forking or Similar Mechanisms*: Downstream organizations can "fork" the main policy repository

or create distinct branches to add local adaptations while syncing upstream improvements.

2.2 The Emergence of Docs-as-Code in Organizational Settings

In traditional documentation workflows, content creators relied on word processors and silo-ed distribution channels. However, large-scale technology firms like Microsoft, Google, and Amazon have long recognized that text-based content—managed as "code"—affords significant advantages in accuracy, version control, and collaboration [5]. This methodology, commonly referred to as *Docs-as-Code*, posits that all documentation (including policy) is best managed in a repository akin to source code, complete with standard best practices such as peer review, continuous integration, and automated testing [6].

Academic discourse supports the notion that flattening hierarchical structures and encouraging more distributed forms of knowledge creation can drive innovation and efficiency [7]. By adopting Code-as-Policy, organizations effectively flatten the policy development process, allowing for continuous and participatory feedback loops—an approach aligned with agile governance principles [8].

2.3 Code-as-Policy vs. Policy Documentation

The Department of Defense's extensive portfolio of instructions and directives is more than simple "documentation." These documents establish rules, protocols, and procedures that have legal and operational implications. "Code-as-Policy" can operate at two levels:

1. **Textual Level**: The actual verbiage of DoDIs and directives is stored in plain text (Markdown, AsciiDoc, etc.) with version control.
2. **Enforcement Level (Optional/Advanced)**: Certain policy statements are translated into machine-readable code that can be automatically checked or enforced by policy engines (e.g., security configurations, data handling procedures).

While the focus of this paper is primarily on the textual level (the immediate step for policy management), further integration with machine-enforceable policy opens powerful avenues for compliance automation [9]. For instance, one might encode a policy on approved software libraries into a scanning tool that rejects unapproved code. Having policy in structured, version-controlled formats also facilitates extraction by AI systems, enabling automated analytics, semantic search, and real-time policy updates based on changing operational contexts.

3 The Case for a Version-Controlled Code-as-Policy Model

3.1 Enhanced Transparency and Accountability

Traceability and "Blame" With a modern version control system, every single revision is linked to its originator, timestamp, and reason for the change. This full historical lineage solves one of the biggest challenges in traditional document-based systems: identifying precisely who changed what and when. Such transparency is critical in an organization as large and complex as the Department of Defense, particularly when accountability and oversight are paramount [10].

Review and Approval Branch-based (or stream-based) workflows enable formal pull/merge requests, facilitating peer reviews and sign-offs by subject matter experts (SMEs), legal teams, and

command leadership. This ensures that policy changes are thoroughly vetted before they become official.

Navigability, Cross-Linking, and Issue Tracking Beyond simply tracking who changed what, version control platforms (e.g., GitHub, GitLab, or Bitbucket) offer advanced features that significantly enhance transparency and usability:

1. **Cross-Repository References:** Directives in one repository can link to related guidance in another repository, eliminating the fragmentation caused by scattered PDFs. This interconnected approach allows policy authors and readers to navigate from one directive to another with a single click, ensuring that all stakeholders consistently reference the most up-to-date versions.
2. **Snapshot Policy State:** Each commit constitutes a permanent, timestamped "snapshot" of the policy, enabling the Department of Defense to roll back to or audit any historical state with precision. Legal teams, oversight bodies, and leadership can reconstruct the exact policy environment in effect at critical decision points or incidents.
3. **Commit and Pull Request History:** Proposed changes (pull requests) capture the rationale, code (or text) diffs, and reviewer comments in one place. This creates a transparent narrative around each policy update, clarifying how wording evolved, who approved it, and why certain revisions were made.
4. **Issue Tracking and Collaboration:** Instead of emailing tracked changes, participants open "issues" to highlight ambiguities, note conflicts with other directives, or propose enhancements. Each discussion is recorded publicly (or within a secured environment for classified policy), providing a robust feedback loop that continuously improves policy clarity and consistency.

Together, these mechanisms transform the static nature of Department of Defense directives into an interactive, auditable knowledge graph, where every link, comment, and change is fully visible, reviewable, and recoverable.

3.2 Faster Iteration and Continuous Improvement

Reduction of "Version Confusion" In legacy systems, multiple versions of the same document circulate simultaneously, often creating confusion about which version is authoritative. By using a single authoritative "main" branch (or "trunk"), the canonical state of the policy is always clear [11].

Synchronized Local Adaptations Subordinate commands can fork or branch from the main repository to adapt policies for local contexts without losing alignment with upstream changes. This fosters a more agile, bottom-up approach to policy evolution.

3.3 Automation, Integration, and AI Readiness

CI/CD Pipelines Just as code changes are automatically tested, policy changes can trigger automated builds that produce PDF, HTML, or other desired formats. Automated quality checks (spelling, style, broken links, compliance with style guides) and security checks (e.g., scanning for inclusion of sensitive data) can be integrated into the pipeline [12].

Machine-Readable Extensions and AI Pipelines Future enhancements could embed metadata and structured rules within policy text, enabling automation of compliance checks against operational configurations (e.g., verifying that cybersecurity protocols align with NIST standards). These structured representations are also conducive to modern AI pipelines. By parsing policy text from a single version-controlled source, AI-driven systems can:

1. Perform *natural language understanding* and semantic analysis on up-to-date policy documents.
2. Provide *real-time decision support* by mapping policy statements to relevant operational data.
3. *Flag potential ambiguities* or conflicts in policy through machine learning models trained on historical policy changes and adjudication outcomes.

Such synergy between policy text, automation, and AI is a foundational principle of DevSecOps [13], now evolving into AI-augmented governance.

3.4 Cultural and Organizational Benefits

Empowerment of Personnel A Code-as-Policy model can democratize policy contributions, allowing individuals at various levels to propose edits or clarifications. While final approvals remain hierarchical, the open and transparent environment encourages a culture of continuous improvement [14].

Breaking Down Silos Centralized repositories reduce duplication of effort across different Department of Defense branches and units. Shared development fosters cross-domain collaboration, ultimately enhancing uniformity and interoperability within the Department of Defense.

3.5 Long-Term Strategic Alignments

Digital Transformation Embracing modern version control in policy development aligns with broader Department of Defense initiatives for digital modernization, in which the organization must continuously adapt to the swiftly changing threat landscape and technological environment [15].

Future-Proofing Plaintext-based policy stores and open file formats mitigate issues related to vendor lock-in or obsolescence. They also scale seamlessly as the volume and complexity of policy artifacts grow. Moreover, having policy in standardized markup formats will facilitate future transitions to advanced AI systems that ingest, interpret, and enforce policy at scale.

4 Potential Challenges and Mitigation Strategies

4.1 Cultural Resistance and Change Management

Challenge Department of Defense personnel may be accustomed to Word-PDF workflows and might see version control systems (especially Git) as overly technical or unfamiliar. Additionally, the hierarchical structure and entrenched workflows within the Department of Defense can create significant barriers to adopting new methodologies.

Mitigation — Comprehensive Change Management

1. **Incentivization Metrics**: Develop metrics that reward departments and individuals for adopting and effectively using the CaP model. This could include recognition programs, performance evaluations, and linking adoption to professional development opportunities.
2. **Resistance from Mid-Level Leadership**: Engage mid-level leaders through targeted training and demonstrate the tangible benefits of CaP. Establish champions within these leadership tiers to advocate for the transition and address concerns.
3. **Structured Training Programs**: Implement ongoing training initiatives tailored to different roles within the DoD. Provide hands-on workshops, online tutorials, and support resources to ease the transition.
4. **Pilot Programs**: Launch pilot programs within progressive divisions to showcase the effectiveness of the CaP model. Use success stories from these pilots to build momentum and encourage broader adoption across the organization.

Mitigation — Automated Multi-Format Publishing With a Code-as-Policy approach, continuous integration/continuous deployment (CI/CD) pipelines can automatically generate policy artifacts in multiple outputs—PDF, static websites, Word documents, and more—from a single canonical source. This lets Department of Defense personnel continue using familiar PDF or Word formats, while also making policy updates available on internal portals or statically generated websites. The result is consistent formatting, fewer manual conversion errors, and faster propagation of approved changes.

4.2 Security and Classification

Challenge Some policy documents contain sensitive or classified information, posing significant risks if version control repositories are not properly secured.

Mitigation — Deeper Security Measures

1. **Secure Hosting**: Host repositories on Department of Defense-accredited platforms (e.g., a private GitLab or Perforce instance within a secure enclave).
2. **Access Controls**: Implement robust role-based permissions and multi-factor authentication, ensuring only authorized personnel can view or modify sensitive content.
3. **Encryption and Zero-Trust Principles**: Encrypt data both at rest and in transit. Employ zero-trust network architecture to continuously verify user and service identities [17].
4. **Segmentation**: Partition classified and unclassified policies into separate repositories, ensuring that the highest levels of classification remain insulated.

4.3 Governance, Compliance, and Legal Considerations

Challenge The formal policy approval process in the Department of Defense can be bureaucratic. Ensuring that version-controlled changes comply with existing legal frameworks and records management is crucial.

Mitigation — Enhanced Governance and Compliance Framework

1. **Customized Pull Request Flow**: Automate a final approval step requiring sign-off from legal counsel, records management officers, and commanding officers. This ensures that all policy changes undergo necessary legal and compliance reviews before being merged.
2. **Extended Metadata**: Embed classification labels, Freedom of Information Act (FOIA) disclaimers, and revision reasons directly in policy files. Automated scripts can enforce the presence of mandatory metadata, ensuring compliance with Department of Defense Instruction (DoDI) 5015.02 on records management [18].
3. **Integration with Official Records Management Systems**: Use APIs to synchronize approved policy states with systems that comply with DoDI 5015.02. This ensures that all policy documents are properly archived and retrievable in accordance with federal records management standards.
4. **Legal Precedents for Metadata-Enforced Compliance**: Reference legal frameworks and precedents that support the use of metadata for enforcing compliance. This includes citing cases where metadata has been effectively used to track and enforce policy adherence.

These strategies not only ensure compliance with existing legal and governance frameworks but also enhance the credibility and reliability of the CaP model within the Department of Defense.

4.4 Scalability Across a Large Organization

Challenge The Department of Defense's massive structure spans multiple military departments, combatant commands, and agencies. Achieving uniform adoption is difficult when each entity has distinct requirements.

Mitigation — Managing Decentralization

1. **Tiered Rollout Strategy**: Start with smaller commands or specialized working groups (e.g., software factories) to pilot the approach. Gradually expand to larger and more diverse commands, incorporating lessons learned and best practices from initial rollouts.
2. **Centralized Governance, Decentralized Execution**: Maintain a core "main" repository that sets overarching policy. Allow each department to maintain branches or forks for local adaptations, reconciling changes upstream through scheduled merges.
3. **Shared Best Practices Repository**: Create a knowledge base of reusable workflows, training materials, and scripts that can be adapted across different branches of the Department of Defense. This repository serves as a centralized resource to standardize practices and facilitate efficient scaling.

Mitigation — Feasibility Analysis and Pilot Program Metrics Conduct a thorough feasibility analysis to address scalability challenges unique to the Department of Defense's size and complexity. This includes:

1. **Integration with Classified Networks**: Develop strategies for integrating CaP with secure networks like SIPRNet, ensuring that classified policies are managed with the highest security standards.
2. **Resource Investments**: Outline the necessary personnel, infrastructure, and training resources

required for a successful rollout. This includes identifying key roles, budgeting for infrastructure upgrades, and allocating time for training programs.

3. **Cost-Benefit Analysis**: Perform a cost-benefit analysis to demonstrate the financial and operational advantages of adopting the CaP model. Highlight potential savings from reduced document management inefficiencies and improved policy agility.
4. **Pilot Program Metrics**: Establish metrics to evaluate the success of pilot programs, such as adoption rates, user satisfaction, reduction in policy update times, and compliance levels. Use these metrics to guide iterative improvements and inform broader deployment strategies.

4.5 Technical Infrastructure and Interoperability

Challenge Many Department of Defense components rely on legacy systems like SharePoint or specialized intranets, complicating integration.

Mitigation — Hybrid Models and APIs

1. **API Bridges and Webhooks**: Develop bridging software that syncs version-controlled documents with SharePoint libraries, ensuring that end-users see the latest "published" policy versions.
2. **Document Synchronization Pipelines**: Automate PDF or HTML generation from the repository, then push these artifacts to existing intranets or portals [19].
3. **Embrace Hybrid Solutions**: Some Department of Defense components might initially require a partial Docs-as-Code approach, retaining certain Word/PDF workflows where absolutely necessary while still benefiting from version control on critical policy text.

4.6 User-Centric Design and Accessibility for Non-Technical Stakeholders

The successful adoption of the Code-as-Policy model requires engagement not only from technical and policy-making personnel but also from a broader audience, including non-technical staff who interact with policies daily. Many of these users may be unfamiliar with version control systems or the underlying methodologies that support the CaP framework. Without adequate consideration for their needs, the initiative risks creating barriers that could hinder widespread adoption and effectiveness.

4.6.1 Challenges for Non-Technical Stakeholders

1. **Steep Learning Curve**: Traditional workflows involving Word and PDF documents are deeply ingrained, and transitioning to version-controlled repositories may appear daunting for non-technical users.
2. **Complex Interfaces**: Tools like Git and associated platforms (e.g., GitHub, GitLab) are designed primarily for developers, which can make navigation and interaction overwhelming for non-technical users.
3. **Role-Specific Needs**: Non-technical roles such as administrative staff, compliance officers, and field personnel may require tailored workflows that differ significantly from those used by developers or policymakers.

4.6.2 Proposed Mitigation Strategies

1. **User-Friendly Interfaces**:
   - Develop or integrate graphical user interfaces (GUIs) tailored to non-technical users. These interfaces should simplify complex actions such as version navigation, pull requests, and branch management into intuitive, guided workflows.
   - Example: A web-based portal that abstracts the technical details of version control while providing clear options for document review, commenting, and approvals.
2. **Simplified Workflows**:
   - Implement templates and automated scripts to reduce manual effort for repetitive tasks. For instance, policy review workflows could be streamlined to allow users to approve changes via email links or web forms without interacting directly with a version control platform.
   - Pre-configure commonly used actions (e.g., creating new branches for local adaptations) to reduce decision fatigue and errors.
3. **Comprehensive Training Programs**:
   - Develop tailored training modules for non-technical users that focus on practical, role-specific applications of the CaP model. These modules should:
     – Use non-technical language and relatable examples.
     – Include hands-on demonstrations of user-friendly interfaces.
     – Provide ongoing support through help desks or dedicated training personnel.
   - Incentivize participation in training programs by linking them to professional development credits or recognition awards.
4. **Accessible Documentation**:
   - Create clear, jargon-free documentation that explains the system's purpose, workflows, and benefits. Ensure that this documentation is easily accessible in various formats, including video tutorials and FAQs.
   - Example: Interactive user guides embedded within the policy platform to offer real-time assistance.
5. **Feedback Loops**:
   - Actively solicit feedback from non-technical users to refine interfaces, workflows, and training programs. Establish a mechanism for continuous improvement based on their experiences and challenges.
   - Example: Regular surveys or user experience workshops to identify pain points and address them iteratively.

4.6.3 Anticipated Benefits

1. **Broader Adoption**: Lowering the entry barrier for non-technical users ensures that the CaP model is embraced across all levels of the organization, maximizing its impact.
2. **Enhanced Collaboration**: Simplified workflows and interfaces foster collaboration between technical and non-technical stakeholders, creating a more inclusive policy development process.

3. **Improved Compliance**: By making the version-controlled repositories accessible and understandable to a wider audience, the organization can ensure consistent compliance with directives across all operational areas.

By prioritizing user-centric design and accessibility, the Code-as-Policy initiative can achieve greater inclusivity and adoption. Tailoring tools, workflows, and training to non-technical stakeholders ensures that the benefits of the CaP model are realized across the entire Department of Defense enterprise.

4.7 AI Risks and Mitigations

Artificial intelligence (AI) and machine learning hold transformative potential for policy management within the Department of Defense. However, their implementation must address key risks to ensure responsible and effective use.

4.7.1 Ethical Risks

1. **Bias in AI-Generated Policies**: AI systems trained on historical data may inadvertently perpetuate existing biases, resulting in unfair, unrealistic, inefficient or detrimental policy outcomes.
2. **Opacity of Decision-Making**: AI-driven processes can lack transparency, making it difficult to discern how policy recommendations are derived and reducing trust in automated decisions.

4.7.2 Technical Constraints

1. **NLP Accuracy**: Natural Language Processing (NLP) tools may misinterpret the complexity and nuance of policy language, leading to errors in automated analyses.
2. **Data Quality Dependence**: AI systems require comprehensive, well-structured data. Incomplete or inconsistent data inputs can undermine the reliability of AI-generated insights.

4.7.3 Mitigation Strategies

1. **Bias Mitigation**: Utilize diverse training datasets and implement techniques to identify and reduce bias in AI models.
2. **Explainable AI**: Adopt AI models that provide transparent decision-making processes, enabling policymakers to understand and trust AI-generated recommendations.
3. **Continuous Validation**: Regularly evaluate AI tools against real-world policy scenarios to ensure their accuracy and reliability.
4. **Robust Data Governance**: Establish stringent data management practices to maintain the quality, consistency, and completeness of data inputs for AI systems.

Additionally, forming an **AI Ethics Board** within the Department of Defense can oversee the development and deployment of AI tools, ensuring they adhere to ethical standards and organizational values.

4.8 Long-Term Strategic Alignments

Digital Transformation Embracing modern version control in policy development aligns with broader Department of Defense initiatives for digital modernization, in which the organization must continuously adapt to the swiftly changing threat landscape and technological environment [15].

Future-Proofing Plaintext-based policy stores and open file formats mitigate issues related to vendor lock-in or obsolescence. They also scale seamlessly as the volume and complexity of policy artifacts grow. Moreover, having policy in standardized markup formats will facilitate future transitions to advanced AI systems that ingest, interpret, and enforce policy at scale.

5 Modernization Initiatives and the Code-as-Policy Framework

Transitioning from theoretical foundations to practical implementation, it is essential to contextualize the Code-as-Policy model within the Department of Defense's broader modernization efforts. Initiatives such as Joint All-Domain Command and Control (JADC2) and the adoption of Zero Trust architectures underscore the necessity for agile, scalable, and secure policy management systems. The CaP framework aligns seamlessly with these initiatives by providing a robust, version-controlled approach to policy development that supports real-time adaptability and comprehensive integration across various domains.

JADC2 emphasizes the integration and synchronization of data across all domains-air, land, sea, space, and cyber-to achieve a unified operational picture. The CaP model complements this by ensuring that policy documents are consistently updated, easily accessible, and seamlessly integrated into operational workflows. Similarly, Zero Trust architectures, which operate on the principle of "never trust, always verify," benefit from the granular access controls and auditability inherent in version-controlled policy repositories.

By embedding the CaP framework within these modernization initiatives, the Department of Defense can enhance its operational readiness, ensure policy compliance across all domains, and foster a culture of continuous improvement and innovation.

6 Implementation Framework

Implementing the Code-as-Policy paradigm within the Department of Defense necessitates a structured and strategic approach. The Implementation Framework delineates the phased rollout, governance model, training and change management, and supporting tools and automations essential for a successful transition. This section provides a comprehensive roadmap, addressing both technical and organizational facets to ensure scalability, security, and sustained adoption across the Department of Defense.

6.1 Phased Rollout

A phased rollout strategy is paramount to manage the complexity and scale of the Department of Defense's policy infrastructure. This approach minimizes risks, allows for continuous learning, and facilitates stakeholder buy-in at each stage.

1. **Pilot Phase:**

- **Selection of Flagship Policy:** Identify a flagship Department of Defense Instruction (DoDI) for initial conversion to the CaP model. Criteria for selection should include the policy's strategic importance, frequency of updates, and potential for cross-departmental impact.
- **Formation of a Cross-Functional Team:** Assemble a small, diverse team comprising policy experts, software engineers, DevSecOps practitioners, and change management specialists. This team will spearhead the pilot, develop best practices, configure necessary tools, and create initial training modules.
- **Development of Best Practices and Tool Configurations:** Establish standardized procedures for policy conversion, including version control workflows, branching strategies, and code review protocols. Configure tools such as Git repositories, CI/CD pipelines, and automated validation scripts tailored to the Department of Defense's requirements.
- **Initial Training and Documentation:** Develop comprehensive training materials and conduct workshops to familiarize the pilot team with the CaP tools and methodologies. Documentation should cover repository management, pull request workflows, and security protocols.
- **Evaluation and Feedback Collection:** Monitor the pilot's progress, gather feedback from team members, and identify challenges. Use these insights to refine processes and tools before scaling.

2. **Limited Production:**
   - **Expansion to Related Policies:** Gradually extend the CaP model to a limited set of related policies within the same domain or operational area. This ensures consistency and leverages lessons learned from the pilot phase.
   - **Integration of Lessons Learned:** Incorporate feedback and refine best practices based on pilot outcomes. Address any identified gaps in tooling, training, or governance.
   - **Enhanced Documentation and Support:** Expand training materials to accommodate the broader set of policies and provide additional support resources, such as help desks or dedicated support teams, to assist new adopters.
   - **Stakeholder Engagement and Communication:** Maintain transparent communication with all stakeholders, highlighting successes and addressing concerns. Use pilot successes to build momentum and demonstrate the tangible benefits of the CaP model.
3. **Full Deployment:**
   - **Department-Wide Rollout:** Deploy the CaP model across the entire Department of Defense, encompassing all relevant policies and directives. Ensure that the rollout is adaptable to the diverse requirements of various commands, branches, and agencies within the Department of Defense.
   - **Formalized Training Programs:** Implement comprehensive, department-wide training initiatives tailored to different roles, including policy authors, reviewers, approvers, and technical support staff. Utilize blended learning approaches combining in-person workshops, online modules, and hands-on labs.
   - **Establishment of Governance Frameworks:** Develop and enforce governance policies that oversee repository management, access controls, compliance standards, and audit procedures. This ensures consistency, security, and accountability across all policy repositories.

- **Continuous Improvement and Iterative Refinement:** Establish mechanisms for ongoing evaluation and improvement of the CaP model. Solicit regular feedback, conduct periodic reviews, and adapt processes to evolving needs and technological advancements.
- **Scalability and Resource Allocation:** Ensure that sufficient resources-both human and technological-are allocated to support the full deployment. This includes scaling infrastructure, expanding support teams, and maintaining robust CI/CD pipelines to handle increased policy volumes.

6.2 Governance Model

A robust governance model is essential to manage the complexities of policy creation, maintenance, and enforcement within a version-controlled environment. The governance model ensures that policies remain consistent, compliant, and aligned with organizational objectives while facilitating collaboration across diverse departments.

1. **Authoritative "Main" Branch:**
   - **Centralized Management:** The main branch serves as the single source of truth for all official policies. It is managed by a central policy office responsible for final approvals and overarching governance.
   - **Strict Access Controls:** Implement role-based access controls to restrict write permissions to authorized personnel, ensuring that only vetted changes are incorporated into the main branch.
   - **Change Management Protocols:** Establish formal procedures for proposing, reviewing, and merging changes. This includes mandatory code reviews, approval workflows, and automated compliance checks.
2. **Review Committees:**
   - **Composition:** Assemble committees comprising Subject Matter Experts (SMEs), legal counsel, senior leadership, and representatives from relevant departments. These committees are responsible for reviewing and approving proposed policy changes.
   - **Structured Review Processes:** Define clear guidelines for review timelines, feedback mechanisms, and decision-making authority. Utilize pull requests and merge requests to facilitate transparent and traceable reviews.
   - **Accountability and Oversight:** Ensure that all changes are thoroughly vetted for legal compliance, operational relevance, and strategic alignment before integration into the main branch.
3. **Subordinate Branches or Forks:**
   - **Decentralized Adaptations:** Allow each command or department to maintain their own branches or forks to handle local adaptations of policies. This decentralization enables tailored updates without disrupting the central policy framework.
   - **Synchronization Mechanisms:** Implement scheduled merges or synchronization protocols to incorporate upstream changes from the main branch into subordinate branches. This ensures that local adaptations remain aligned with overarching policies.
   - **Conflict Resolution Strategies:** Develop procedures for identifying and resolving conflicts

between local adaptations and central policies, promoting harmony and consistency across the organization.

4. **CODEOWNERS Business Rules:**
   - **Designation of Policy Owners:** Utilize "CODEOWNERS" files within the version-controlled repositories to assign specific teams or individuals as owners of distinct policy sections. This designation ensures clear accountability and streamlines the update process.
   - **Asynchronous Updates and Coordination:** By assigning ownership at the section level, different offices can manage and update their respective policy sections independently. This facilitates asynchronous updates, reducing bottlenecks and enhancing agility while maintaining overall coordination.
   - **Integration with Access Controls:** Combine "CODEOWNERS" with access control mechanisms to enforce that only designated owners can approve changes to their respective sections. This integration enhances security and ensures that updates are reviewed by the appropriate stakeholders.
   - **Facilitating Cross-Departmental Collaboration:** Sections of policy that intersect multiple departments can have multiple owners or require joint approvals, ensuring comprehensive review and fostering collaborative governance. For instance, a cybersecurity policy section might be co-owned by the Cyber Operations Office and the Information Assurance Office, necessitating approval from both before changes are merged.

6.3 Training and Change Management

Successful adoption of the CaP model hinges on effective training and change management strategies. These initiatives ensure that personnel are adequately prepared to transition to new workflows and embrace the cultural shift towards collaborative policy management.

1. **Comprehensive Training Programs:**
   - **Role-Based Training:** Develop tailored training modules for different roles, including policy authors, reviewers, approvers, and technical support staff. Each module should focus on the specific responsibilities and workflows pertinent to the role.
   - **Hands-On Workshops and Simulations:** Conduct interactive workshops and simulation exercises to provide practical experience with version control systems, pull request workflows, and policy repository management.
   - **Ongoing Support and Resources:** Establish a repository of training materials, including tutorials, FAQs, and best practice guides. Provide continuous support through help desks, mentorship programs, and dedicated training personnel.
2. **Cultural Onboarding:**
   - **Promoting a Collaborative Mindset:** Emphasize the value of transparency, accountability, and continuous improvement. Highlight success stories and demonstrate how the CaP model enhances operational efficiency and policy agility.
   - **Leadership Endorsement and Advocacy:** Engage senior leadership to champion the CaP initiative, fostering a top-down endorsement that encourages widespread adoption and mitigates resistance.

- **Change Management Frameworks:** Implement structured change management methodologies, such as Kotter's 8-Step Change Model or the ADKAR model, to guide the organization through the transition. This includes creating a sense of urgency, building a guiding coalition, and reinforcing new behaviors.

3. **Incentivization and Recognition:**
   - **Incentivization Metrics:** Develop metrics that reward departments and individuals for adopting and effectively utilizing the CaP model. Metrics could include the number of successful policy updates, adherence to best practices, and contributions to shared repositories.
   - **Recognition Programs:** Establish recognition programs that acknowledge and celebrate exemplary contributions to the CaP initiative. This could include awards, public acknowledgments, and professional development opportunities.
   - **Linking to Performance Evaluations:** Integrate CaP adoption and performance into official performance evaluations, ensuring that contributions to the initiative are valued and incentivized at all organizational levels.
4. **Pilot Programs and Feedback Loops:**
   - **Initial Pilot Programs:** Launch pilot programs within progressive divisions or specialized working groups to showcase the effectiveness of the CaP model. Use these pilots to gather insights, refine processes, and build a repository of best practices.
   - **Iterative Feedback Mechanisms:** Implement continuous feedback loops to collect input from users, identify pain points, and make iterative improvements. This could involve regular surveys, user experience workshops, and dedicated feedback channels.
   - **Scaling Success Stories:** Leverage the successes and lessons learned from pilot programs to inform broader deployment strategies. Highlight tangible benefits, such as reduced policy update times and improved collaboration, to encourage adoption across the organization.

6.4 Supporting Tools and Automations

Leveraging modern tools and automations is critical to operationalizing the CaP model. These technologies streamline workflows, enforce compliance, and enhance the efficiency and reliability of policy management processes.

1. **Integrated CI/CD Pipelines:**
   - **Automated Builds and Deployments:** Implement Continuous Integration/Continuous Deployment (CI/CD) pipelines to automate the building, testing, and deployment of policy documents. This ensures that changes are consistently integrated and deployed with minimal manual intervention.
   - **Policy Linting and Formatting Checks:** Incorporate automated linting and formatting tools to enforce consistency and adherence to predefined style guides. This reduces errors and ensures uniformity across all policy documents.
   - **Automated PDF/HTML Generation:** Configure pipelines to automatically generate multiple output formats, such as PDF and HTML, from the source repository. This facilitates easier distribution and access to policies in various formats.
2. **Metadata and Compliance Checks:**

- **Embedding Metadata:** Integrate metadata fields within policy files to capture essential information such as classification levels, revision history, and compliance requirements. This metadata aids in automated compliance checks and ensures that policies adhere to Department of Defense standards.
- **Automated Compliance Enforcement:** Utilize scripts and tools to automatically verify that all policy documents meet compliance criteria before they are merged into the main branch. This includes checks for mandatory sections, proper classification labels, and adherence to formatting standards.

3. **Dashboards and Monitoring Tools:**
   - **Real-Time Dashboards:** Develop dashboards that visualize key metrics such as open merge requests, pending approvals, and policy coverage across commands. These dashboards provide leadership with a comprehensive overview of the CaP implementation status.
   - **Compliance Status Indicators:** Implement visual indicators that display the real-time compliance status of policies, highlighting areas that require attention or are pending review.
   - **Automated Alerts and Notifications:** Configure alerting mechanisms to notify relevant stakeholders of important events, such as policy updates, failed compliance checks, or upcoming deadlines. This ensures timely action and maintains the momentum of the CaP initiative.
4. **Security Integrations:**
   - **Secure Repository Hosting:** Host policy repositories on secure, Department of Defense-accredited platforms that comply with stringent security standards. This includes using private instances of GitLab, GitHub Enterprise, or other approved version control systems.
   - **Role-Based Access Controls (RBAC):** Implement RBAC to ensure that only authorized personnel can access, modify, or approve policy documents. This minimizes the risk of unauthorized changes and maintains the integrity of policies.
   - **Continuous Security Audits:** Integrate security audit tools into the CI/CD pipelines to continuously monitor and assess the security posture of policy repositories. This includes scanning for vulnerabilities, enforcing encryption standards, and ensuring compliance with Zero Trust principles.

7 From Theory to Practice: Azure DevOps for Code-as-Policy

In line with the Code-as-Policy model, this whitepaper itself has been developed using a structured and automated workflow facilitated by Azure DevOps. This approach ensures that the document remains version-controlled, auditable, and consistently formatted, embodying the principles it advocates for. This section outlines the infrastructure and processes employed to build, validate, and deploy this whitepaper, demonstrating the practical application of the CaP model.

Figure 1 illustrates the end-to-end Azure DevOps pipeline configured to build, validate, and deploy the whitepaper. Each stage is interconnected, ensuring seamless transitions from code commits to final deployment, while integrating both automated and manual approval gates to maintain quality and compliance. Detailed technical configurations and scripts are provided in Appendix 12.

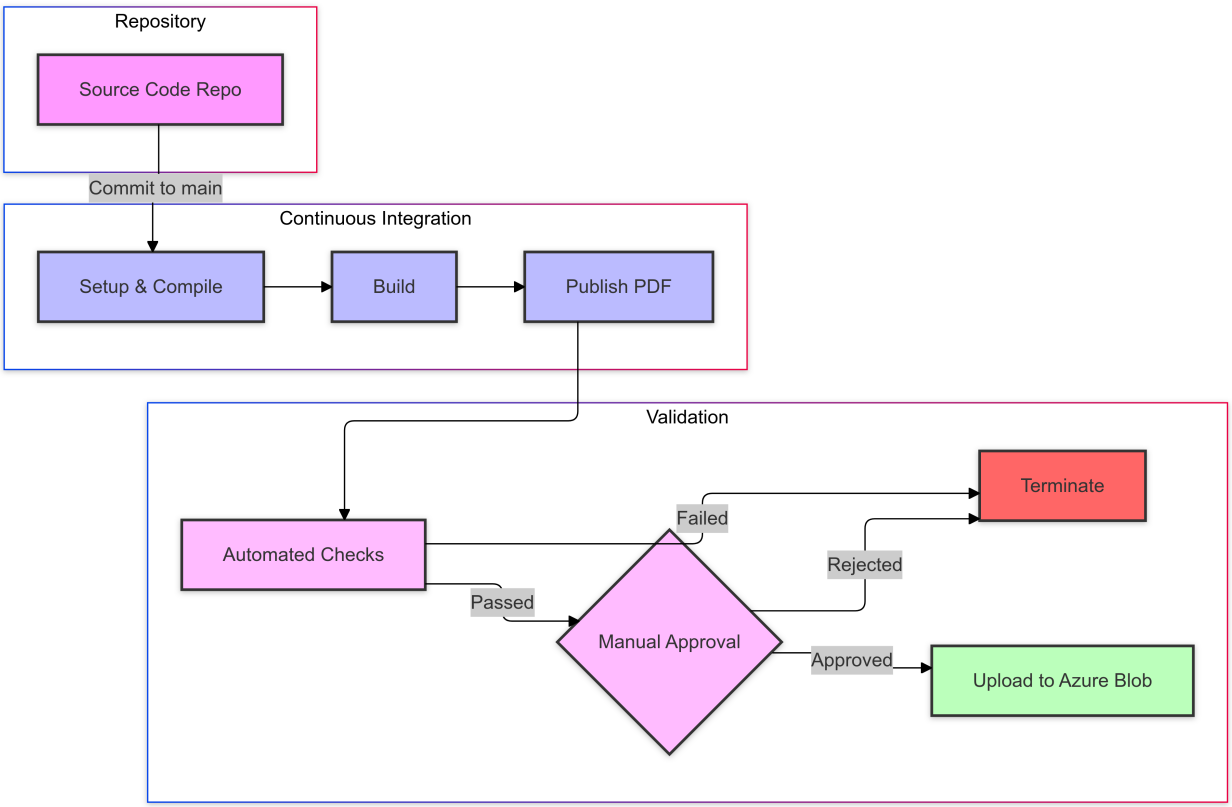


Figure 1: Code-as-Policy Azure DevOps Pipeline Workflow

7.1 Repository Management

At the heart of our automated workflow is a dedicated Azure DevOps repository that houses all the source files necessary for the whitepaper's development. The repository structure is organized to maintain clarity and efficiency:

- **Root Directory**: Contains the main LaTeX file (‘CodeAsPolicy.tex‘), ‘.gitignore‘, ‘README.md‘, ‘azure-pipelines.yml‘, and ‘azure-pipelines-1.yml‘.
- **Subdirectories**:
  - ‘images/‘: Stores all diagrams and figures referenced in the whitepaper.
  - ‘references/‘: Holds bibliography files and citation databases.

7.2 .gitignore Configuration

To maintain a clean repository, a ‘.gitignore‘ file is configured to exclude extraneous files generated during the LaTeX build process. The following ‘.gitignore‘ rules are applied:

```
# LaTeX build and auxiliary files
*.aux
*.fdb_latexmk
*.fls
*.log
```

```
*.out
*.synctex.gz
*.toc

# Ignore PDF files in the root directory
/*.pdf

# Allow PDF files in all subdirectories
!**/*.pdf

```

This configuration ensures that only essential source files are tracked, while build artifacts like ‘.aux‘, ‘.log‘, and root-level PDFs are ignored. PDFs generated within subdirectories (e.g., ‘build/output.pdf‘) are preserved for deployment purposes.

7.3 README.md: Onboarding and Documentation

The ‘README.md‘ file serves as the primary entry point for contributors and stakeholders, providing an overview of the project, setup instructions, and guidelines for collaboration. Key components include:

- Project Description: Brief summary of the whitepaper’s purpose and objectives.
- Getting Started: Instructions on how to clone the repository, install dependencies, and build the document locally.
- Contribution Guidelines: Best practices for submitting changes, including branching strategies, pull request protocols, and review processes.
- Licensing Information: Details about the project’s licensing terms.
- Author Information: Contact details and professional background of the author(s).

This structured documentation ensures that new contributors can quickly become productive and understand the workflow established within the repository.

7.4 Automated Build Pipeline

Azure DevOps pipelines are configured to automate the build and validation processes, ensuring that every commit is systematically processed and validated. The pipeline is divided into two primary stages: **Build** and **Validation**.

7.4.1 Build Stage

The Build stage is responsible for compiling the LaTeX source files into a PDF document. It includes the following steps, outlined in Appendix 12.1.1:

1. **Caching LaTeX Installation**: Utilizes Azure DevOps’ caching mechanism to store the LaTeX installation, significantly reducing build times for subsequent runs.
2. **Installing Minimal LaTeX**: Installs only the necessary LaTeX packages required for building the whitepaper, optimizing storage and build times.
3. **Compiling LaTeX Document**: Uses ‘latexmk‘ to automate the multi-pass compilation process,

generating a PDF in the designated ‘output‘ directory.

4. **Publishing PDF as Artifact**: The generated PDF is published as a pipeline artifact named ‘PDF‘, making it available for subsequent stages.

7.4.2 Validation Stage

The Validation stage performs a series of automated checks to ensure the integrity and compliance of the whitepaper before deployment. It comprises two jobs: AutomatedChecks and ManualApproval.

Automated Checks Job This job executes a series of scripts to verify the presence of critical sections within the LaTeX source file, ensuring that the document adheres to the expected structure and content standards. Additionally, it integrates AI-based analysis to assess the document's correctness, coherence, and adherence to professional standards.

1. **Section Validations**:
   - **Table of Contents**:

```
- script: |
  if grep -q '\\tableofcontents' $(System.DefaultWorkingDirectory)/$(TEX_FILE);
    then
    echo "Table of Contents found."
  else
    echo "Error: Table of Contents not found."
    exit 1
  fi
  displayName: "Check for Table of Contents"

```

   - **Abstract Section**:

```
- script: |
  if grep -q '\\section\*\s*{Abstract}' $(System.DefaultWorkingDirectory)/$(
    TEX_FILE); then
    echo "Abstract section found."
  else
    echo "Error: Abstract section not found."
    exit 1
  fi
  displayName: "Check for Abstract Section"

```

   - **Title Section**:

```
- script: |
  if grep -q '\\title{' $(System.DefaultWorkingDirectory)/$(TEX_FILE); then
    echo "Title section found."
  else
    echo "Error: Title section not found."
    exit 1
```

```
    fi
    displayName: "Check for Title Section"

```

- **References/Bibliography Section**:

```
- script: |
    if grep -q '\\bibliography{' $(System.DefaultWorkingDirectory)/$(TEX_FILE);
      then
      echo "References/Bibliography section found."
    else
      echo "Error: References/Bibliography section not found."
      exit 1
    fi
    displayName: "Check for References/Bibliography Section"

```

- **Conclusion Section**:

```
- script: |
    if grep -q '\\section{Recommendations}\label{recommendations}

\begin{enumerate}
    \item \textbf{Establish Code-as-Policy as the default for policy change} by
      requiring all new and revised policies to have a canonical repository, a
      change log, and pull-request review.
    \item \textbf{Instrument policy with testable controls} (linters, schema checks
      , and automated compliance assertions) so policy quality is enforced
      continuously, not at publication time.
    \item \textbf{Adopt a paved road for policy delivery} (templates, CI pipelines,
       and approval workflows) to reduce friction and make the secure path the easy
       path.
    \item \textbf{Define governance boundaries} (who can approve what, escalation
      paths, and audit requirements) so speed increases without sacrificing
      accountability.
    \item \textbf{Pilot with one high-churn policy family in 60 days} and measure
      cycle time, defect rate, and adoption across stakeholders.
\end{enumerate}

\section{Conclusion}' $(System.DefaultWorkingDirectory)/$(TEX_FILE); then
      echo "Conclusion section found."
    else
      echo "Error: Conclusion section not found."
      exit 1
    fi
    displayName: "Check for Conclusion Section"

```

2. **AI-Based Validation**:

- Integrates the OpenAI API to analyze the PDF for correctness, coherence, and adherence to

predefined standards.

- This step uses a custom Python script to interact with OpenAI's services, which assigns a pass or fail rating based on the analysis results. An Azure Function or other serverless architecture could also be employed to host this functionality if needed.
- The below task calls the AI-based analysis script as part of the pipeline:

```
- task: Bash@3
  displayName: "AI-Based PDF Analysis with OpenAI"
  inputs:
    targetType: 'inline'
    script: |
      python analyze_ai.py --pdf $(System.DefaultWorkingDirectory)/output/
     CodeAsPolicy.pdf
  continueOnError: false # Fail if non-zero code

```

Manual Approval Deployment Job This deployment job is associated with the 'Production' environment in Azure DevOps, configured with manual approval gates. Upon passing all automated checks, the pipeline pauses here, awaiting human approval before proceeding to deployment.

```
- deployment: ManualApproval
  displayName: "Manual Approval Deployment"
  dependsOn: AutomatedChecks
  environment: 'Production'  # Use the environment created in Azure DevOps
  strategy:
    runOnce:
      deploy:
        steps:
        - script: echo "Awaiting manual approval..."
          displayName: "Manual Approval Placeholder"

```

This structure ensures that policy deployments undergo thorough scrutiny, combining automated integrity checks with human oversight to maintain the highest standards of quality and compliance.

7.5 Deployment Pipeline

The final stage of the pipeline involves deploying the validated PDF to a designated Azure Blob Storage container, making it accessible to authorized stakeholders.

1. **Download PDF Artifact**:
   - Retrieves the published PDF artifact from the Build stage.
2. **Upload to Azure Blob Storage**:
   - Utilizes the 'AzureCLI' task to upload the PDF to the specified Azure Blob Storage container.
   - Ensures secure handling of connection strings and credentials through Azure DevOps' secure variables.

```yaml
- stage: Deploy
  displayName: "Deploy PDF"
  dependsOn: Validation
  jobs:
  - job: DeployJob
    displayName: "Deploy PDF to Azure Blob Storage"
    steps:
    # Download the PDF Artifact
    - task: DownloadPipelineArtifact@2
      displayName: "Download PDF Artifact"
      inputs:
      artifact: "PDF"
      path: "$(System.DefaultWorkingDirectory)/downloaded_pdf"

    # Debugging step to list files
    - script: |
      echo "Listing contents of the downloaded_pdf directory:"
      ls -R $(System.DefaultWorkingDirectory)/downloaded_pdf
      displayName: "Debug: Verify Downloaded Artifacts"

    # Deploy to Azure Blob Storage
    - task: AzureCLI@2
      displayName: "Upload PDF to Azure Blob Storage"
      inputs:
      azureSubscription: "AzureBlobConnection"
      scriptType: 'bash'
      scriptLocation: 'inlineScript'
      inlineScript: |
        az storage blob upload \
          --account-name $(DEPLOY_STORAGE_ACCOUNT) \
          --container-name $(DEPLOY_CONTAINER) \
          --name CodeAsPolicyFromPDFstoPullRequests.pdf \
          --file $(System.DefaultWorkingDirectory)/downloaded_pdf/
    CodeAsPolicyFromPDFstoPullRequests.pdf \
          --connection-string "$(DEPLOY_CONNECTION_STRING)"

```

Note: For the policy contributor to ensure that the ‘CodeAsPolicy.pdf‘ filename matches the actual generated PDF name in the build process, the debugging step is added to view file paths and names in the logs more easily.

8 Broader Impact, Real-World Examples, and Future Research

8.1 Digital Bureaucracy and Agile Governance

By flattening the hierarchy of policy creation, the Department of Defense can evolve into a more agile bureaucracy that swiftly adapts to changing geopolitical and technological landscapes. Real-time policy updates can be tested, audited, and merged in days or weeks instead of months or years, accelerating the Department of Defense’s responsiveness.

8.1.1 Real-World Implementations in Government Contexts

1. **NASA (Internal Documentation)**: Transitioned portions of mission documentation to Docs-as-Code, reducing errors in final documents and improving collaboration across departments.
2. **UK Government Digital Service (GDS)**: Migrated web guidance and policy content to Git-based platforms, decreasing publication time by an order of magnitude and improving version traceability.
3. **Estonia's e-Government Infrastructure**: Though not purely Git-based, Estonia relies heavily on automated, version-controlled processes for e-services, showcasing how digital policy management can enhance service delivery at scale.

8.2 AI-Augmented Policy and Decision Support

Structured policy repositories are a natural fit for AI and machine learning tools. By leveraging these technologies, the Department of Defense can transform policy management into a dynamic, adaptive process that integrates real-time insights and predictive capabilities. This subsection explores key applications of AI in augmenting policy and decision support:

1. **Semantic Linking and Knowledge Graphs**: AI algorithms can construct knowledge graphs that interlink policy topics, directives, and relevant operational data, creating a rich, interconnected repository of information. These graphs enable:
   - Context-Rich Decision Support: By connecting policy provisions with real-time data sources, such as intelligence feeds or operational metrics, decision-makers gain deeper insights into how policies influence and are influenced by ongoing events.
   - Streamlined Information Retrieval: Knowledge graphs allow users to navigate from a high-level policy directive to specific operational guidelines or historical precedents, significantly improving situational awareness and reducing time spent searching for information.
2. **Automated Compliance Monitoring**: Intelligent agents embedded within the policy repositories can continuously monitor systems and processes for compliance with established directives. These agents can:
   - Real-Time Alerts: Automatically flag non-compliant configurations, such as deviations from cybersecurity protocols or data handling requirements, enabling swift corrective actions.
   - Dynamic Policy Updates: Detect emerging risks or changes in mission requirements and suggest updates to relevant policies, ensuring that guidance remains aligned with current needs.
   - Historical Audits: Provide detailed records of compliance over time, aiding in accountability and transparency during reviews or investigations.
3. **Predictive Impact Analyses**: Leveraging machine learning models, predictive analytics can assess the potential outcomes and unintended consequences of proposed policy changes. This capability can:
   - Risk Assessment: Highlight areas of potential vulnerability or conflict before a policy is implemented, allowing for preemptive mitigation strategies.
   - Scenario Simulation: Model various operational scenarios under different policy conditions, giving decision-makers a data-driven basis for selecting the most effective course of action.

- Learning from Historical Data: Use past policy outcomes to refine models, improving the accuracy and relevance of predictions over time.

4. **Policy Recommendation Systems**: Advanced AI systems, such as large language models and natural language processing tools, can assist policymakers by:
   - Drafting Policy Language: Generating initial drafts based on provided objectives and constraints, which can then be reviewed and refined by human experts.
   - Identifying Gaps: Analyzing the policy repository to detect inconsistencies, redundancies, or omissions, ensuring a more cohesive and comprehensive policy framework.
   - Proposing Alternatives: Suggesting alternative approaches based on historical data, industry best practices, or simulated outcomes.
5. **Adaptive Threat Intelligence Integration**: By integrating real-time threat intelligence feeds, AI systems can dynamically evaluate policies against evolving threat landscapes. For example:
   - Zero-Day Vulnerabilities: Quickly flag and adapt policies in response to newly discovered cybersecurity vulnerabilities.
   - Mission-Specific Adjustments: Tailor policy guidance to reflect the unique risks and requirements of specific missions or operations.
   - Proactive Defense: Use predictive models to identify and address potential threats before they materialize, ensuring continuous operational readiness.

These AI-enabled capabilities offer transformative potential for the Department of Defense, enabling a policy ecosystem that is not only responsive to change but also anticipates and mitigates challenges. By integrating AI into policy management, the Department can achieve greater agility, accountability, and precision, ensuring its policy framework remains a robust pillar of operational success.

8.3 Policy Enforcement via Machine-Readable Code

While this paper focuses on textual policy, advanced implementations can encode key policy clauses into machine-readable rules. Policy engines like *Open Policy Agent* or custom Department of Defense solutions can enforce these rules automatically:

1. **Security Configurations**: Real-time scanning ensures systems adhere to policy-defined baseline configurations.
2. **Data Handling Procedures**: Automated pipelines reject data flows that violate classification or privacy stipulations.
3. **Procurement Requirements**: Smart contracts or embedded policy logic can verify contractor compliance before approving transactions.

These examples illustrate the leap from mere documentation management to proactive and automated compliance.

8.4 Adaptive Threat Intelligence Integration

As global threat vectors rapidly evolve, particularly in cyberspace, the Department of Defense must continuously adapt its policies to remain effective. By leveraging AI-driven threat intelligence platforms in concert with version-controlled policy repositories, the Department of Defense can integrate real-time insights from diverse data sources—cyber threat feeds, sensor networks, and

intelligence analysis—directly into policy management pipelines.

Dynamic Risk Management Automated frameworks could evaluate emerging threats against current directives and propose swift policy updates where discrepancies or vulnerabilities arise. For instance, if an AI system detects a new zero-day software exploit affecting critical systems, it could flag relevant sections of DoDI guidance for revision, launch a pull request in the policy repository, and alert the appropriate stakeholders to expedite review and approval.

Operational Advantages

1. **Shorten Reaction Times**: Rapidly patch operational guidelines or security protocols based on new intelligence.
2. **Improve Coordination**: Align policy updates across multiple commands and agencies in real-time, reducing fragmentation.
3. **Facilitate Proactive Measures**: Leverage historical intelligence and predictive analytics to anticipate emerging threat vectors and preemptively adjust policy before incidents occur.

8.5 Imaginative Future Directions

The Code-as-Policy paradigm is not merely a technical shift but a foundational transformation with the potential to unlock groundbreaking innovations in policy creation, management, and enforcement. Several forward-looking possibilities merit exploration to push the boundaries of what a policy ecosystem can achieve:

1. **Neural Policy Advisors as Collaborative Think Tanks**: Leveraging Large Language Models (LLMs) trained on historical policy data, operational insights, and global best practices, neural policy advisors could serve as real-time collaborators. These AI systems would not only suggest new policy language but also simulate potential impacts across various scenarios, detect conflicting directives, and even recommend adaptations to align with evolving mission needs. Imagine a policy drafter asking, "How does this proposal align with cybersecurity directives?" and receiving an instant, AI-driven analysis complete with recommendations and supporting data.
2. **Self-Enforcing Policies with Blockchain-Driven Smart Contracts**: Policies governing procurement, supply chain integrity, or compliance with regulations could be codified into blockchain-based smart contracts. These self-executing contracts would monitor transactions in real time, ensuring that only those adhering to Department of Defense rules proceed. For instance, a procurement policy could automatically block payments to vendors failing to meet security standards or environmental criteria, ensuring continuous enforcement without manual intervention.
3. **Dynamic Cross-Domain Orchestration**: Policies stored in version-controlled repositories could act as triggers for interconnected workflows across domains such as logistics, finance, and human resources. For example:
   - An updated policy on resource prioritization could automatically adjust allocations in enterprise planning systems, synchronizing budgets and personnel assignments across commands.
   - A cybersecurity directive could dynamically reconfigure network access controls or deploy

patches across the enterprise in response to emerging threats.

- Mission-specific adjustments could cascade through systems, ensuring every operational node is aligned with the latest strategic objectives.

4. **Policy Simulation Environments for Real-World Testing**: Advanced simulation environments could allow policymakers to test proposed directives in virtual "digital twins" of Department of Defense operations. By simulating how changes affect mission readiness, compliance, and interdepartmental workflows, leaders could identify gaps, conflicts, or unintended consequences before implementing policies in the real world. Such simulations could also incorporate AI and machine learning models to forecast long-term impacts and assess risk.
5. **Autonomous Policy Adaptation**: Building on AI's potential, future systems could monitor operational data and adjust policies autonomously within predefined thresholds. For example, if intelligence data signals an increase in cyberattacks, an AI system could preemptively tighten access controls, recommend temporary measures, or prioritize relevant directives for human review—all while maintaining a full audit trail.
6. **Decentralized Policy Marketplaces**: A decentralized, blockchain-enabled policy marketplace could allow Department of Defense departments to share, adapt, and exchange modular policy components. Departments could "license" well-vetted policies, adapting them to local contexts while contributing improvements back to the central repository. This would create a collaborative ecosystem where the best ideas rise to the top through collective refinement.

These imaginative directions reflect the transformative potential of the CaP model when combined with cutting-edge technologies like AI, blockchain, and advanced simulation. Pursuing these innovations would position the Department of Defense at the forefront of digital governance, ensuring agility, resilience, and strategic dominance in an era of rapid change.

8.6 Prioritizing Future Research

While imaginative visions foster innovation, prioritization of the following research areas can yield near-term impact:

1. **Ethical and Legal Considerations of AI in Policy**: Investigate how AI-generated policy recommendations interface with existing legal frameworks and ethical guidelines, ensuring transparency and accountability.
2. **Sociotechnical Shifts**: Study how hierarchical structures adapt when policy creation is democratized through pull requests, exploring the long-term cultural and organizational impacts.
3. **Practical Policy Simulation Environments**: Develop simulation environments to test new policies in virtual exercises, leveraging AI/ML to predict real-world outcomes and identify potential conflicts or gaps.
4. **Automated Cross-Referencing and Classification**: Explore advanced NLP techniques to classify and interlink Department of Defense policies, directives, and regulations, providing a holistic view of the entire Department of Defense policy landscape.

9 On the Naming of "Code-as-Policy"

The designation *Code-as-Policy* (CaP) was intentionally chosen to emphasize a human-centric and organizationally transformative approach, distinguishing it from the existing *Policy-as-Code* paradigm. While *Policy-as-Code* typically refers to machine-readable and enforceable rules within automation systems, CaP focuses on managing policy through collaborative, version-controlled processes akin to software development practices.

This nomenclature aligns with the broader *Everything-as-Code* (EaC) philosophy, which seeks to unify various "as-code" methodologies-such as *Infrastructure-as-Code*, *Configuration-as-Code*, and *Docs-as-Code*-into an interoperable ecosystem. By positioning policy management within this framework, CaP underscores the shift from static, document-based policies to dynamic, adaptable repositories that enhance transparency, accountability, and collaboration.

Furthermore, the term CaP avoids conflating this initiative with existing technical frameworks, thereby fostering clear differentiation and reducing potential confusion among stakeholders. This distinction highlights CaP's unique focus on leveraging version control for policy development and management, promoting a culture of continuous improvement and agility within the Department of Defense.

In summary, *Code-as-Policy* encapsulates the initiative's intent to revolutionize policy management by integrating it into the software development lifecycle, thereby enhancing its responsiveness and alignment with modern technological and organizational demands.

10 Conclusion

Migrating to a version-controlled, Code-as-Policy paradigm offers the Department of Defense a transformative solution to the persistent challenges of policy versioning, accountability, and responsiveness. By embracing the principles of Docs-as-Code, the DoD can convert static policy documents into dynamic, living repositories that reflect ongoing learning, collaboration, and adaptation across the department. This transition not only enhances transparency and accountability but also positions the DoD at the forefront of digital governance, ensuring operational excellence and strategic agility in an era dominated by rapid technological advancements and evolving global threats.

Despite the technical, cultural, and governance challenges inherent in such a transformation, a phased rollout, robust governance frameworks, comprehensive training, and strategic use of advanced technologies like AI and blockchain provide a clear pathway to successful implementation. The Code-as-Policy model is not merely a technical upgrade but a foundational shift towards a more agile, accountable, and AI-ready Department of Defense.

Future research and continuous innovation will further refine and expand the capabilities of the CaP model, ensuring its alignment with the DoD's evolving needs and the broader landscape of digital modernization. By committing to this paradigm shift, the Department of Defense can achieve unprecedented levels of policy management efficiency, security, and adaptability, thereby safeguarding its mission and maintaining strategic dominance in an increasingly complex world.

11 Glossary

AI-augmented Governance The integration of artificial intelligence technologies to enhance decision-making, policy enforcement, and operational efficiencies within governance frameworks.

Agile Governance A framework that applies Agile methodologies to governance processes, promoting flexibility, iterative development, and continuous improvement in policy management and decision-making.

Artificial Intelligence (AI) A branch of computer science focused on creating systems capable of performing tasks that typically require human intelligence, such as understanding natural language, recognizing patterns, and making decisions.

Autonomous Systems Systems that operate independently without human intervention, utilizing sensors, machine learning, and decision-making algorithms to perform tasks. In the DoD context, this includes drones, robotics, and unmanned vehicles.

Branching and Merging Version control operations that allow for the creation of separate lines of development (branches) and the integration of changes from different branches (merging) into a unified codebase.

CI/CD (Continuous Integration/Continuous Deployment) A set of practices in software development where code changes are automatically tested and deployed, ensuring rapid and reliable delivery of software updates.

Code-as-Policy (CaP) A paradigm where organizational policies are managed as version-controlled code, enabling dynamic updates, collaboration, and integration with development workflows.

Configuration-as-Code (CaC) The management of system configurations through machine-readable files, allowing for version control, automation, and reproducibility.

Continuous Integration (CI) A software development practice where developers frequently merge their code changes into a central repository, followed by automated builds and tests to detect issues early.

Continuous Deployment (CD) An extension of Continuous Integration where code changes are automatically deployed to production environments after passing automated tests, ensuring rapid and reliable release cycles.

Cyber-Physical Operations Activities that integrate computational (cyber) systems with physical processes, enabling coordinated control and interaction between software-driven technologies and the physical environment. In the Department of Defense context, this includes the use of autonomous systems, drones, robotics, and sensor networks to achieve strategic and tactical objectives securely and efficiently.

DevOps A set of practices that combines software development (Dev) and IT operations (Ops), aiming to shorten the development lifecycle and provide continuous delivery with high software quality.

DevSecOps An extension of DevOps that integrates security practices into the software development and operations lifecycle, ensuring security is a shared responsibility.

Docs-as-Code The practice of managing documentation using version control systems and software development tools, facilitating collaboration, versioning, and automation.

Digital Transformation The integration of digital technology into all areas of a business, fundamentally changing how organizations operate and deliver value to customers. In the DoD, this involves modernizing infrastructure, processes, and policies to enhance efficiency and effectiveness.

Infrastructure-as-Code (IaC) The management of infrastructure (networks, servers, storage) through code and automation, enabling consistent and scalable deployments.

JADC2 (Joint All-Domain Command and Control) A Department of Defense initiative aimed at integrating and synchronizing data across all military domains to enhance operational effectiveness.

Knowledge Graphs Structured representations of knowledge that capture entities, their attributes, and the relationships between them. Knowledge graphs enable enhanced data integration, search capabilities, and semantic understanding.

Machine Learning (ML) A subset of AI focused on developing algorithms that allow computers to learn from and make predictions or decisions based on data without being explicitly programmed for specific tasks.

Open Policy Agent An open-source, general-purpose policy engine that enables unified, context-aware policy enforcement across the entire stack. It allows organizations to define policies in a high-level declarative language.

Pull Requests A feature in version control systems where contributors propose changes to a codebase, which are then reviewed and approved by maintainers before integration.

Quantum Computing A type of computing that takes advantage of quantum phenomena, such as superposition and entanglement, to perform operations on data. Quantum computing has the potential to solve complex problems much faster than classical computers.

Smart Contracts Self-executing contracts with the terms of the agreement directly written into code. Smart contracts automatically enforce and execute contractual clauses when predefined conditions are met, often utilizing blockchain technology.

Version Control Systems (VCS) Tools that track changes to files over time, allowing multiple collaborators to work on a project simultaneously while maintaining a history of modifications. Examples include Git, Perforce, and Subversion.

YAML (YAML Ain't Markup Language) A human-readable data serialization standard commonly used for configuration files and data exchange between languages with different data structures.

Zero Trust Architectures A security model that assumes no implicit trust granted to assets or user accounts based solely on their physical or network location, requiring continuous verification.

12 Appendix: Supporting Scripts

This section provides implementation details for the AI-based validation described in the pipeline. Below is the Python script used to analyze the generated PDF for correctness, coherence, and adherence to standards using the OpenAI API.

```python
import argparse
import openai
from PyPDF2 import PdfReader
import sys

# OpenAI API key
openai.api_key = "YOUR_OPENAI_API_KEY"

def extract_text_from_pdf(pdf_path):
  """Extracts text from a PDF file."""
  reader = PdfReader(pdf_path)
  text = ""
  for page in reader.pages:
    text += page.extract_text()
  return text

def analyze_with_openai(text):
  """Analyzes for coherence, correctness, and adherence to standards."""
  prompt = f"""
  You are a document analysis expert. Analyze the following text for:
  1. Correctness, are there any factual errors or unclear statements?
  2. Coherence, is the document well-organized and easy to follow?
  3. Adherence to professional standards, does it align with writing norms?
  Provide a pass/fail verdict with a very concise explanation.

  Document Text:
  {text[:3000]}  # Limiting to 3000 characters.
  """

  response = openai.ChatCompletion.create(
    model="gpt-4",
    messages=[{"role": "user", "content": prompt}]
  )
  return response["choices"][0]["message"]["content"]

def main(pdf_path):
  print(f"Analyzing PDF: {pdf_path}")
  text = extract_text_from_pdf(pdf_path)
  if not text:
    print("Failed to extract text from the PDF. Exiting.")
    sys.exit(1)

  analysis = analyze_with_openai(text)
  print("AI Analysis Result:")
```

```
  print(analysis)

  # Determine pass/fail from analysis
  if "pass" in analysis.lower():
    print("AI verdict: PASS")
    sys.exit(0)  # Exit with success
  else:
    print("AI verdict: FAIL")
    sys.exit(1)  # Exit with failure

if __name__ == "__main__":
  parser = argparse.ArgumentParser(description="Analyze.")
  parser.add_argument("--pdf", required=True, help="Path.")
  args = parser.parse_args()
  main(args.pdf)

```

12.1 Detailed Azure DevOps Pipeline Configurations

For readers interested in the granular technical details of the Azure DevOps pipeline configurations, the following YAML snippets provide comprehensive guidance. These configurations are integral to automating the build, validation, and deployment processes as outlined in Section 6.

12.1.1 azure-pipelines.yml

```yaml
# Azure DevOps Pipeline for building a PDF from LaTeX
trigger:
  branches:
    include:
      - main

pool:
  vmImage: 'ubuntu-latest'

variables:
  TEX_FILE: "CodeAsPolicyFromPDFstoPullRequests.tex"
  OUTPUT_DIR: "output"

steps:
- task: Cache@2
  displayName: "Cache LaTeX Installation"
  inputs:
    key: "latex-ubuntu-$(Agent.OS)-$(Agent.OSArchitecture)"
    path: "/usr/share/texlive"
    restoreKeys: |
      latex-ubuntu

# Debugging step to verify file locations
- script: |
```

```
    echo "Workspace contents:"
    ls -R $(System.DefaultWorkingDirectory)
  displayName: "Debug: List Workspace Contents"

# Install LaTeX
- task: Bash@3
  displayName: "Install Minimal LaTeX"
  condition: ne(variables['CacheRestored'], 'true')  # Only install if cache not restored
  inputs:
    targetType: 'inline'
    script: |
      sudo apt-get update
      sudo apt-get install -y texlive-latex-recommended texlive-latex-extra texlive-fonts-recommended latexmk

# Compile LaTeX to PDF
- task: Bash@3
  displayName: "Compile LaTeX Document"
  inputs:
    targetType: 'inline'
    script: |
      mkdir -p $(System.DefaultWorkingDirectory)/$(OUTPUT_DIR)
      latexmk -pdf -output-directory=$(System.DefaultWorkingDirectory)/$(OUTPUT_DIR) $(System.DefaultWorkingDirectory)/$(TEX_FILE)

# Publish PDF artifact
- task: PublishPipelineArtifact@1
  displayName: "Publish PDF Artifact"
  inputs:
    targetPath: "$(System.DefaultWorkingDirectory)/$(OUTPUT_DIR)"
    artifact: "PDF"

```

12.1.2 azure-pipelines-1.yml

```
# Azure DevOps Pipeline for consuming the built PDF
trigger: none

resources:
  pipelines:
    - pipeline: InternalBuild  # Alias for the first pipeline
      source: "Whitepaper.CodeAsPolicy Internal Build"  # Exact name of the first pipeline in Azure DevOps
      trigger:
        branches:
          include:
```

```yaml
            - main  # Triggers this pipeline when the 'main' branch of the first pipeline is updated

pool:
  vmImage: 'ubuntu-latest'

variables:
  DEPLOY_STORAGE_ACCOUNT: "wpcodeaspolicy"   # Azure Storage Account Name
  DEPLOY_CONTAINER: "whitepapers"           # Blob Container Name
  DEPLOY_CONNECTION_STRING: "$(AZURE_STORAGE_CONNECTION_STRING)"  # Azure Storage Connection String (set as a secret variable)

stages:
  # Stage 1: Validation
  - stage: Validation
    displayName: "Validate PDF"
    jobs:
      - job: AutomatedChecks
        displayName: "Run Automated Checks"
        steps:
          # Download the PDF Artifact from the first pipeline
          - task: DownloadPipelineArtifact@2
            displayName: "Download PDF Artifact"
            inputs:
              buildType: 'specific'  # Correct parameter
              project: '$(System.TeamProject)'  # Assumes same project
              pipeline: ${{ resources.pipelines.InternalBuild.pipelineId }}  # Reference the pipeline ID
              runVersion: 'latest'  # Fetch the latest run
              artifact: 'PDF'  # Name of the artifact
              path: '$(Pipeline.Workspace)/downloaded_pdf'  # Destination path

          # Debugging step to verify downloaded artifact
          - script: |
              echo "Listing contents of the downloaded_pdf directory:"
              ls -R $(Pipeline.Workspace)/downloaded_pdf
            displayName: "Debug: Verify Downloaded Artifacts"

          # Validation Steps
          - script: |
              echo "Contents of the LaTeX file:"
              cat $(Pipeline.Workspace)/downloaded_pdf/CodeAsPolicyFromPDFstoPullRequests.tex
            displayName: "Debug: Print LaTeX File"

          - script: |
              if grep -q '\\tableofcontents' $(Pipeline.Workspace)/downloaded_pdf/CodeAsPolicyFromPDFstoPullRequests.tex; then
                echo "Table of Contents found."
              else
```

```
                echo "Error: Table of Contents not found."
                exit 1
              fi
            displayName: "Check for Table of Contents"

          - script: |
              if grep -q '\\section\*\s*{Abstract}' $(Pipeline.Workspace)/
      downloaded_pdf/CodeAsPolicyFromPDFstoPullRequests.tex; then
                echo "Abstract section found."
              else
                echo "Error: Abstract section not found."
                exit 1
              fi
            displayName: "Check for Abstract Section"

          - script: |
              if grep -q '\\title{' $(Pipeline.Workspace)/downloaded_pdf/
      CodeAsPolicyFromPDFstoPullRequests.tex; then
                echo "Title section found."
              else
                echo "Error: Title section not found."
                exit 1
              fi
            displayName: "Check for Title Section"

          - script: |
              if grep -q '\\bibliography{' $(Pipeline.Workspace)/downloaded_pdf/
      CodeAsPolicyFromPDFstoPullRequests.tex; then
                echo "References/Bibliography section found."
              else
                echo "Error: References/Bibliography section not found."
                exit 1
              fi
            displayName: "Check for References/Bibliography Section"

          - script: |
              if grep -q '\\section{Conclusion}' $(Pipeline.Workspace)/downloaded_pdf/
      CodeAsPolicyFromPDFstoPullRequests.tex; then
                echo "Conclusion section found."
              else
                echo "Error: Conclusion section not found."
                exit 1
              fi
            displayName: "Check for Conclusion Section"

          % AI-Based Validation (Details in Appendix)
          - task: Bash@3
            displayName: "AI-Based PDF Analysis with OpenAI"
            inputs:
              targetType: 'inline'
```

```
              script: |
                python analyze_ai.py --pdf $(Pipeline.Workspace)/downloaded_pdf/CodeAsPolicyFromPDFstoPullRequests.pdf
              continueOnError: false # Fail if non-zero code

      % Manual Approval Deployment Job
      - deployment: ManualApproval
        displayName: "Manual Approval Deployment"
        environment: 'Production'  % Ensure this environment is set up in Azure DevOps
        dependsOn: AutomatedChecks
        strategy:
          runOnce:
            deploy:
              steps:
                - script: echo "Awaiting manual approval..."
                  displayName: "Manual Approval Placeholder"

  % Stage 2: Deploy
  - stage: Deploy
    displayName: "Deploy PDF"
    dependsOn: Validation
    jobs:
      - job: DeployJob
        displayName: "Deploy PDF to Azure Blob Storage"
        steps:
          # Download the PDF Artifact from the first pipeline
          - task: DownloadPipelineArtifact@2
            displayName: "Download PDF Artifact"
            inputs:
              buildType: 'specific'  # Correct parameter
              project: '$(System.TeamProject)'  # Assumes same project
              pipeline: ${{ resources.pipelines.InternalBuild.pipelineId }}  # Reference the pipeline ID
              runVersion: 'latest'  # Fetch the latest run
              artifact: 'PDF'  # Name of the artifact
              path: '$(Pipeline.Workspace)/downloaded_pdf'  # Destination path

          # Debugging step to verify downloaded artifact
          - script: |
              echo "Listing contents of the downloaded_pdf directory:"
              ls -R $(Pipeline.Workspace)/downloaded_pdf
            displayName: "Debug: Verify Downloaded Artifacts"

          # Deploy to Azure Blob Storage
          - task: AzureCLI@2
            displayName: "Upload PDF to Azure Blob Storage"
            inputs:
              azureSubscription: "AzureBlobConnection"  # Ensure this service connection is configured
              scriptType: 'bash'
```

```
149                scriptLocation: 'inlineScript'
150                inlineScript: |
151                  az storage blob upload \
152                    --account-name $(DEPLOY_STORAGE_ACCOUNT) \
153                    --container-name $(DEPLOY_CONTAINER) \
154                    --name CodeAsPolicyFromPDFstoPullRequests.pdf \
155                    --file $(Pipeline.Workspace)/downloaded_pdf/
       CodeAsPolicyFromPDFstoPullRequests.pdf \
156                    --connection-string "$(DEPLOY_CONNECTION_STRING)"
157
```

References

References

[1] Adam Boas. (2024). *LinkedIn Profile*. Retrieved from https://www.linkedin.com/in/adamnboas/

[2] Wright, S., & Martin, K. (2018). *Docs-as-Code: How Version Control Transforms Documentation Management. Journal of Technical Writing, 34*(2), 177–194.

[3] Loeliger, J., & McCullough, M. (2012). *Version Control with Git*. O'Reilly Media.

[4] Spinellis, D. (2020). Docs in Repos: Lessons from Open Source. *Communications of the ACM, 63*(4), 21–23.

[5] Kitchin, R. (2014). *The Data Revolution: Big Data, Open Data, Data Infrastructures and Their Consequences.* SAGE Publications.

[6] Lamport, L. (2018). Documenting Complex Systems: Why We Adopted Docs-as-Code. *IEEE Software, 35*(3), 14–18.

[7] Morrison, J. (2017). The Open Organization. *Management Science Quarterly, 16*(1), 102–117.

[8] Conboy, K., & Fitzgerald, B. (2010). Method and developer characteristics for effective agile method tailoring: A study of expert opinion. *ACM Transactions on Software Engineering and Methodology, 20*(1), 4–18.

[9] Hashimoto, M. (2017). *Infrastructure as Code: Managing Servers in the Cloud.* O'Reilly Media.

[10] Department of Defense. (2023). *DoD Directive 5100.01: Roles and Responsibilities.* Washington, DC.

[11] Appleton, B., Berczuk, S., & Brown, A. (2020). *Continuous Integration: Improving Software Quality and Reducing Risk.* Addison-Wesley Professional.

[12] Fowler, M., & Highsmith, J. (2001). The Agile Manifesto. *Software Development Magazine, 9*(8), 28–35.

[13] Kim, G., Humble, J., Debois, P., & Willis, J. (2016). *The DevOps Handbook.* IT Revolution Press.

[14] Carucci, R. (2019). Leading Through Change. *Harvard Business Review Press.*

[15] Department of Defense. (2019). *Digital Modernization Strategy.* Washington, DC.

[16] Kotter, J. P. (1996). *Leading Change.* Harvard Business Review Press.

[17] National Institute of Standards and Technology. (2020). *NIST SP 800-53 Rev. 5: Security and Privacy Controls for Information Systems and Organizations.* Gaithersburg, MD.

[18] Department of Defense. (2021). *DoD Instruction 5015.02: Records Management Program.* Washington, DC.

[19] Atlassian Corporation. (2022). *Managing Documentation with CI/CD: Integrations and Best Practices.* Retrieved from https://www.atlassian.com

[20] Halpern, J. Y., & Weissman, V. (2008). Using first-order logic to reason about policies. *ACM Transactions on Information and System Security, 11*(4), 21–42.